IO DOKUMENTACJA

1.Macierz kompetencji zespołu

KOMPETENCJE	PIOTR KURBIEL	ANTONI JANEK	ARKADIUSZ BARAN	FRANCISZEK HAJDUK
FRONTEND	Posiada	Nie posiada	Posiada	Posiada
BACKEND	Nie posiada	Posiada	Nie posiada	Nie posiada
BAZY DANYCH	Posiada	Nie posiada	Posiada	Posiada
IMPLEMENTACJA LLM	Posiada	Posiada	Posiada	Posiada
TESTOWANIE OPROGRAMOWANIA	Nie posiada	Nie posiada	Posiada	Nie posiada
DOKUMENTACJA	Posiada	Posiada	Nie posiada	Nie posiada

2.Pytania dotyczące szczegółów projektu.

Pytania:	Odpowiedzi:	Uwagi:
Czy rozmowa ma być przeprowadzana przez okno tekstowe, czy w formie mowy	Rozmowa powinna zostać przeprowadzona w tekstowym oknie dialogowym (chat)	
Czy aplikacja powinna posiadać szczególne języki które muszą się w niej znaleźć	Aplikacja musi zawierać Języki: Polski, Angielski, Hiszpański	
Czy przeprowadzana rozmowa powinna być zapisywana na koncie użytkownika	Tak, do rozmów można wracać w dowolnej chwili	Potrzeba implementacji logowania
Czy użytkownik aplikacji może sam dostosować poziom zaawansowania rozmowy	Tak, użytkownik wybiera poziom języka	
Czy aplikacja może funkcjonować w formie okna przeglądarki	Tak	Aplikacja desktopowa również dopuszczalna

3. Ustalony format danych wejściowych

Sesje	**Wiadomości**	**Control – warunek**	**Report (Statystyki)**
user_id (int unique const) **session_id** (int unique const) **language** (varchar - np. 'EN', 'ES', 'PL') **level** (varchar - np. 'A1', 'B2', 'C1')	**timestamp** (epoch) **msg_id** (int unique) **sender** (varchar - 'user' lub 'bot') **content** (text) **word_count** (int)	Rozmowa z botem odbywa się ściśle w wybranym języku i na zadeklarowanym poziomie. Po każdej wiadomości użytkownika LLM (lub osobny worker) weryfikuje poprawność, aktualizuje zbiór unikalnych słów i wylicza czas pisania (WPM). Komenda "reset" tworzy nową sesję (nowy session_id) i zeruje kontekst.	**Timestamp** (epoch) - generowany po zakończeniu każdej sesji lub aktualizowany po każdej wiadomości. **wpm** (float - słowa na minutę) **unique_words_count** (int) **total_errors** (int) **errors_details** (jsonb)

4.Baza danych

Tabela: Użytkownicy (Users)

Nazwa kolumny	Typ	Opis
user_id	INT, Primary Key	Unikalny identyfikator użytkownika.
email	VARCHAR, Unique	Adres e-mail użytkownika. Musi być unikalny, służy do logowania oraz odzyskiwania hasła.
password_hash	VARCHAR	Zahaszowane hasło użytkownika
username	VARCHAR	Nazwa użytkownika (nick) wyświetlana w interfejsie aplikacji.
native_language	VARCHAR	Język ojczysty użytkownika
created_at	TIMESTAMP WITH TIME ZONE	Dokładny czas i data założenia konta w systemie.
last_login	TIMESTAMP WITH TIME ZONE	Czas ostatniego udanego logowania

Tabela: Sesje (Sessions)

Nazwa kolumny	Typ	Opis
session_id	INT, Primary Key	Unikalny identyfikator sesji.
user_id	INT, Foreign Key,	Identyfikator użytkownika przypisany do sesji (wymagane do powiązania użytkownika z sesją)
language	VARCHAR	Język konwersacji (np. 'EN', 'ES').
level	VARCHAR	Poziom zaawansowania (np. 'A1', 'B2', 'C1').

Tabela: Wiadomości (Messages)

Nazwa kolumny	Typ	Opis
msg_id	INT, Primary Key	Unikalny identyfikator wiadomości.
session_id	INT, Foreign Key	Relacja do tabeli Sesje (wymagane do powiązania wiadomości z konkretną rozmową).
timestamp	TIMESTAMP WITH TIME ZONE	Dokładny czas utworzenia wiadomości.
sender	VARCHAR	Wskazuje nadawcę: 'user' lub 'bot'.
content	TEXT	Właściwa treść wiadomości.
word_count	INT	Liczba słów w danej wiadomości.

Tabela: Statystyki (Reports)

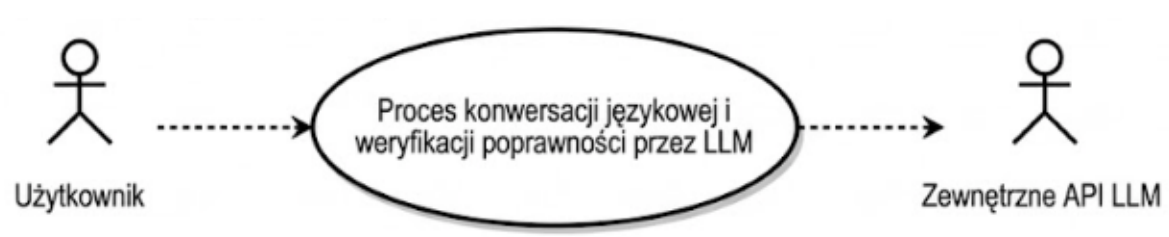
Nazwa kolumny	Typ	Opis
report_id	INT, Primary Key	Identyfikator raportu.
session_id	INT, Foreign Key	Relacja do tabeli Sesje.
timestamp	TIMESTAMP WITH TIME ZONE	Czas wygenerowania po zakończeniu sesji lub czas ostatniej aktualizacji po wiadomości.
wpm	FLOAT	Czas pisania wyrażony w słowach na minutę.
unique_words_count	INT	Liczba unikalnych słów
total_errors	INT	Suma popełnionych błędów.
errors_details	JSONB	Szczegółowe dane o błędach zapisane w formacie JSON.

5. Model Systemu

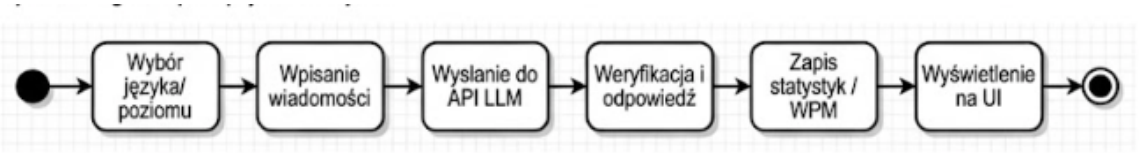
5.1 Model systemu za pomocą tabeli:

Aktorzy	Użytkownik, Zewnętrzne API LLM
Opis	Celem systemu jest interaktywna nauka języka obcego. Użytkownik prowadzi konwersację na wybranym poziomie, a system na bieżąco generuje odpowiedzi i weryfikuje poprawność językową wysyłanych wiadomości.
Dane	Uwierzytelnienie logowania, język docelowy, poziom zaawansowania (np. B2), treść wiadomości, historia sesji.
Wyzwalacz	Przesłanie nowej wiadomości tekstowej przez zalogowanego użytkownika.
Odpowiedź	Odpowiedź tekstowa od bota oraz zwrotny raport z walidacji błędów językowych.
Uwagi	Wymagane połączenie z zewnętrznym API modelu LLM w celu generowania odpowiedzi w czasie rzeczywistym.

5.2 Model systemu za pomocą diagramów UML

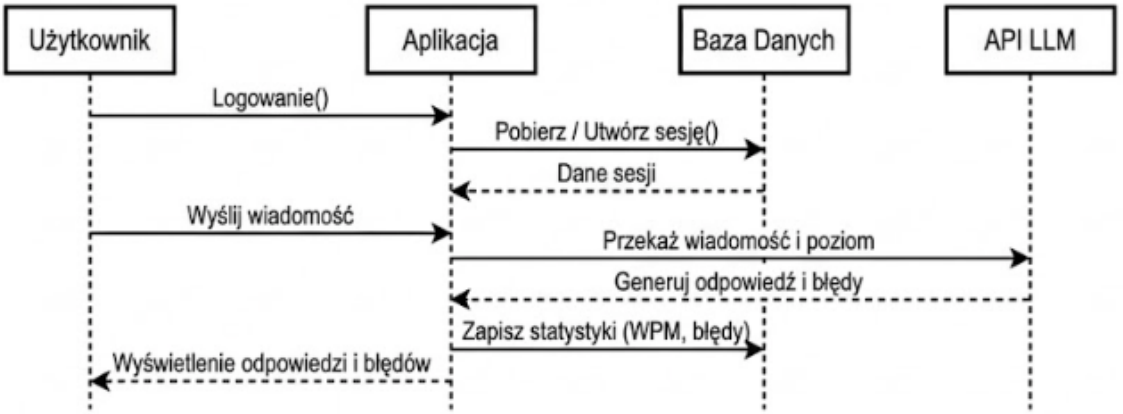


Rys. 1 Diagram przypadków użycia



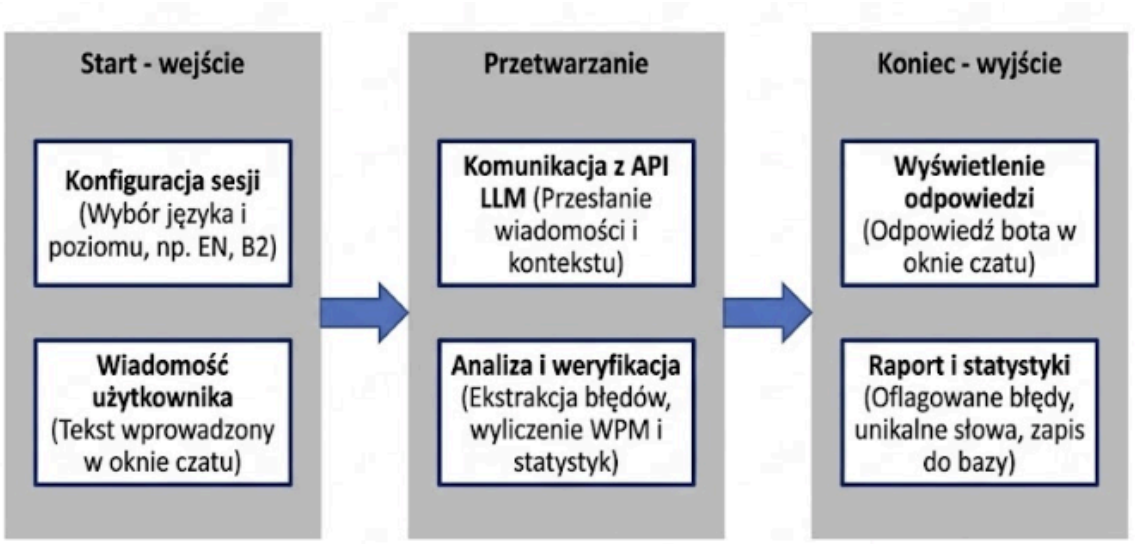
Rys. 2 Diagram przepływu danych

6. Diagram sekwencyjny UML



Rys. 3 Diagram sekwencyjny

7. Projekt architektury opracowywanego systemu



Rys. 4 Architektura systemu

Projekt architektury systemu opiera się na trójetapowym przepływie danych. W fazie wejściowej aplikacja pobiera dane logowania oraz konfigurację sesji, w tym wybrany język i poziom zaawansowania. Następnie rejestruje wprowadzoną wiadomość tekstową użytkownika. W fazie przetwarzania system łączy się z zewnętrznym API modelu LLM, przekazując treść wiadomości wraz z kontekstem rozmowy. Model w czasie rzeczywistym generuje odpowiedź i weryfikuje poprawność gramatyczną, podczas gdy aplikacja wylicza statystyki, takie jak czas pisania (WPM) i unikalne słowa. W fazie wyjściowej gotowa odpowiedź bota oraz oflagowane błędy są natychmiast wyświetlane w interfejsie użytkownika, a zaktualizowane statystyki trafiają bezpośrednio do bazy danych.

8. Sugerowany język implementacji: **Python**

Uzasadnienie: Python jest idealnym wyborem na backend dla tego projektu przede wszystkim ze względu na jego dominację w obszarze sztucznej inteligencji. Większość dostawców modeli LLM oferuje najlepiej zoptymalizowane i najszybciej aktualizowane biblioteki właśnie dla Pythona, co znacznie upraszcza integrację i przetwarzanie zapytań tekstowych. Ponadto nowoczesne frameworki, takie jak FastAPI, natywnie wspierają asynchroniczność, co jest kluczowe dla zachowania płynności czatu podczas oczekiwania na wygenerowanie odpowiedzi przez zewnętrzny model. Python oferuje również doskonałe wsparcie dla bazy PostgreSQL i typu JSONB, co ułatwia zarządzanie statystykami oraz raportami błędów, gwarantując szybki rozwój i dużą stabilność tworzonej aplikacji.

9. Sugerowany LLM implementacji: Google Gemini Flash

Uzasadnienie: Posiada natywne wsparcie dla formatu JSON. Oferuje świetny balans między bardzo niskim kosztem a wysoką jakością analizy wielojęzycznej.

Prompt Systemowy (Instrukcje dla LLM na początku rozmowy):
„Jesteś konwersacyjnym nauczycielem języka docelowego: {Wybrany język sesji}. Językiem ojczystym użytkownika jest: {Język ojczysty użytkownika}. Dostosuj trudność swoich odpowiedzi rygorystycznie do poziomu {Wybrany poziom sesji}. Zwróć wynik wyłącznie jako obiekt JSON w formacie: {"bot_response": "twoja odpowiedź w języku {Wybrany język sesji}", "errors": [{"mistake": "błędny fragment", "correction": "poprawna forma", "explanation": "wyjaśnienie w języku {Język ojczysty użytkownika}"}]}. Każdy błąd w wiadomości użytkownika to osobny element listy errors; jeśli błędów nie ma, zwróć pustą listę.”

Format odpowiedzi jest podany w prompcie wprost, ponieważ bez niego model zwracał wyjaśnienia jako jeden tekst zamiast listy errors i panel „Błędy i poprawa” pozostawał pusty. Dodatkowo zapytanie korzysta z natywnego trybu JSON Gemini (response_mime_type: application/json), który wymusza odpowiedź w postaci poprawnego obiektu JSON.

Używany model: gemini-3.5-flash. Przy chwilowej niedostępności (HTTP 503) kolejne ponowienia trafiają do modeli zapasowych gemini-3.6-flash i gemini-3-flash-preview. Starsze modele (1.5 i 2.5 Flash) nie są już dostępne dla nowych kluczy API.

Aplikacja wysyła do LLM ograniczoną historię rozmowy (3-4 ostatnie wiadomości) dla zachowania ciągłości przy minimalnym zużyciu tokenów. LLM otrzymuje aktualną wiadomość i generuje strukturę wyjściową JSON Schema, która pozwala na łatwą ekstrakcję

liczby błędów do kolumny *total_errors* oraz bezpośredni zapis szczegółów do struktury JSONB.

Struktura wyjściowa:

Nazwa:	Typ:	Opis:
bot_response	String	Właściwa odpowiedź bota w docelowym języku do wyświetlenia w UI.
errors	Array	Lista wykrytych pomyłek. Jeśli wiadomość jest w 100% poprawna, zwraca pustą tablicę
mistake	String	Błędny fragment z tekstu użytkownika. Wysyłane w tablicy *errors*
correction	String	Poprawiona wersja. Wysyłane w tablicy *errors*
explanation	String	Bardzo zwięzłe wyjaśnienie błędu. Wysyłane w tablicy *errors*

10. Sugerowana technologia frontendu: **Streamlit (Python)**

Uzasadnienie: Cały projekt realizujemy w Pythonie, dlatego interfejs użytkownika również powstanie w tym języku. Streamlit pozwala zbudować aplikację webową w samym Pythonie, bez HTML i JavaScript. Posiada gotowe elementy do budowy czatu (*st.chat_message*, *st.chat_input*), do wyświetlania statystyk (*st.metric*) oraz do podziału ekranu na kolumny (*st.columns*). Aplikacja działa w przeglądarce - wersja webowa została zaakceptowana przez klienta. Frontend komunikuje się z backendem (FastAPI) za pomocą zapytań HTTP (biblioteka *requests*) i otrzymuje odpowiedzi w formacie JSON.

Wygląd interfejsu:

Element:	Opis:
Ekran logowania	Logowanie za pomocą adresu e-mail i hasła.
Panel boczny	Wybór języka rozmowy (PL, EN, ES) i poziomu zaawansowania oraz lista poprzednich rozmów, do których można wrócić.

Element:	Opis:
CHAT (lewa część ekranu)	Historia rozmowy (wiadomości użytkownika i bota) oraz pole do wpisania nowej wiadomości.
BŁĘDY I POPRAWA (prawa kolumna)	Dla każdego wykrytego błędu: błędny fragment (*mistake*), poprawna wersja (*correction*) i wyjaśnienie (*explanation*) w języku ojczystym użytkownika.
STATYSTYKI (pod czatem)	Statystyki bieżącej sesji odświeżane po każdej wiadomości - opisane w tabeli poniżej.

Statystyki wyświetlane w interfejsie:

Statystyka:	Sposób wyliczenia:
1. % błędów / zdanie (liczba zdań liczona z treści wiadomości)	*total_errors* / liczba zdań użytkownika × 100%
2. Ilość słów / min	*word_count* / czas między poprzednią odpowiedzią bota a wysłaniem wiadomości (wyliczany z pól *timestamp* w tabeli Wiadomości)
3. Ilość słów wypowiedzianych	Suma *word_count* wszystkich wiadomości użytkownika w sesji
4. Ilość unikalnych słów	*unique_words_count*
5. RESET	Rozpoczęcie nowej sesji (nowy *session_id*), wyczyszczenie czatu i wyzerowanie statystyk. Poprzednia rozmowa pozostaje w historii.

11. Strategia testów i obsługa błędów

11.1 Testy

- **Testy jednostkowe (pytest)** - sprawdzają funkcje liczące statystyki (liczba słów, WPM, unikalne słowa, % błędów) oraz odczyt odpowiedzi JSON z LLM.
- **Testy z atrapą LLM** - w testach backendu zamiast prawdziwego API Gemini używana jest funkcja zwracająca przygotowaną odpowiedź. Dzięki temu testy nie zużywają limitu API, zawsze dają ten sam wynik i pozwalają zasymulować awarię API.
- **Testy ręczne** - scenariusze wykonywane w działającej aplikacji przed oddaniem projektu:

ID:	Test:	Oczekiwany wynik:
TC1	Logowanie poprawnym oraz błędnym hasłem	Wejście do aplikacji / komunikat o błędzie

ID:	Test:	Oczekiwany wynik:
TC2	Nowa rozmowa w języku EN na poziomie B2	Bot odpowiada po angielsku na poziomie B2
TC3	Wysłanie wiadomości bez błędów	Odpowiedź bota, panel błędów pusty
TC4	Wysłanie wiadomości „She go to school yesterday.”	W panelu błąd „go” → „went” z wyjaśnieniem po polsku
TC5	Wysłanie kilku wiadomości	Statystyki zmieniają się po każdej wiadomości
TC6	Kliknięcie RESET	Pusty czat, wyzerowane statystyki, poprzednia rozmowa w historii
TC7	Powrót do poprzedniej rozmowy	Wiadomości wczytane w poprawnej kolejności
TC8	Brak internetu / błąd API	Komunikat o błędzie, aplikacja działa dalej

11.2 Obsługa błędów API LLM i limitowanie zapytań

Sytuacja:	Reakcja systemu:
Przekroczony limit zapytań (HTTP 429) lub chwilowa niedostępność API (HTTP 503)	Maksymalnie 3 ponowienia zapytania z rosnącym odstępem (1 s, 2 s, 4 s). Jeśli wszystkie zawiodą - komunikat „Spróbuj ponownie za chwilę”.
Brak odpowiedzi w ciągu 30 s	Przerwanie zapytania i komunikat dla użytkownika.
Odpowiedź nie jest poprawnym JSON-em	Wyświetlenie samej odpowiedzi bota bez analizy błędów; statystyki błędów nie są aktualizowane.

Dodatkowe zasady:

- Do API wysyłane są tylko 3-4 ostatnie wiadomości (pkt 9), co ogranicza zużycie tokenów.
- Użytkownik może wysłać kolejną wiadomość dopiero po otrzymaniu odpowiedzi bota, a liczba wiadomości jest ograniczona do 10 na minutę, aby nie wyczerpać limitu klucza API.
- W przypadku błędu nic nie jest zapisywane do bazy, a wpisany tekst nie znika - użytkownik może go wysłać ponownie.